Deadlocks

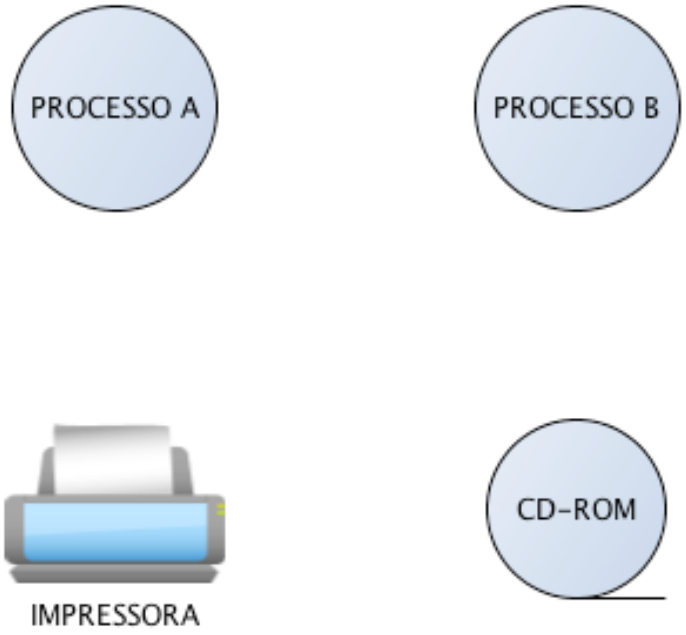
Paulo Valim

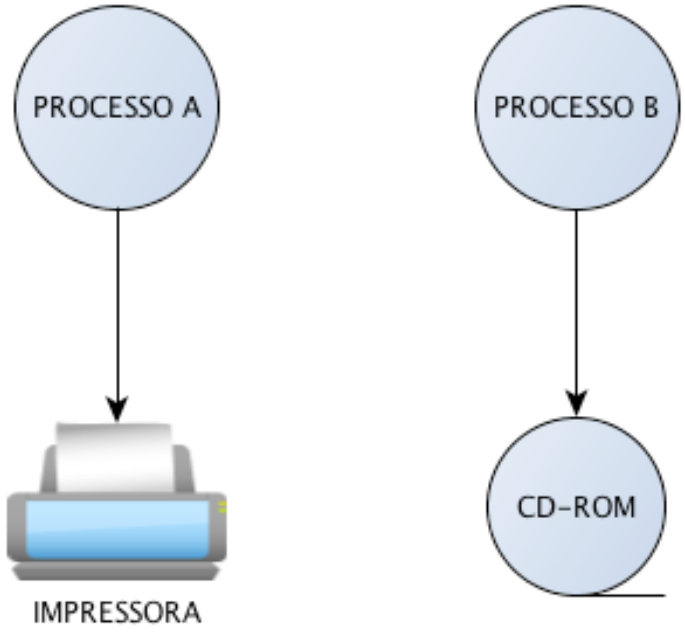
Introdução

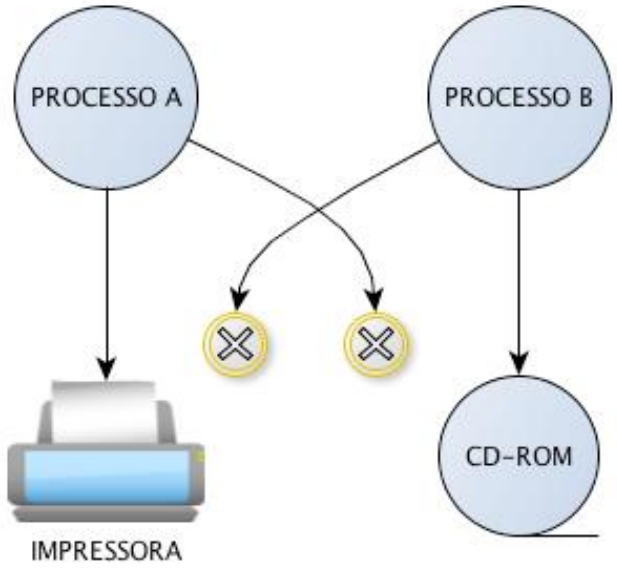
- Sistemas computacionais são cheios de recursos que são usados por um processo de cada vez. Exemplos: impressoras, leitores/gravadores de DVD, unidade de backup, um registro em uma base de dados, etc

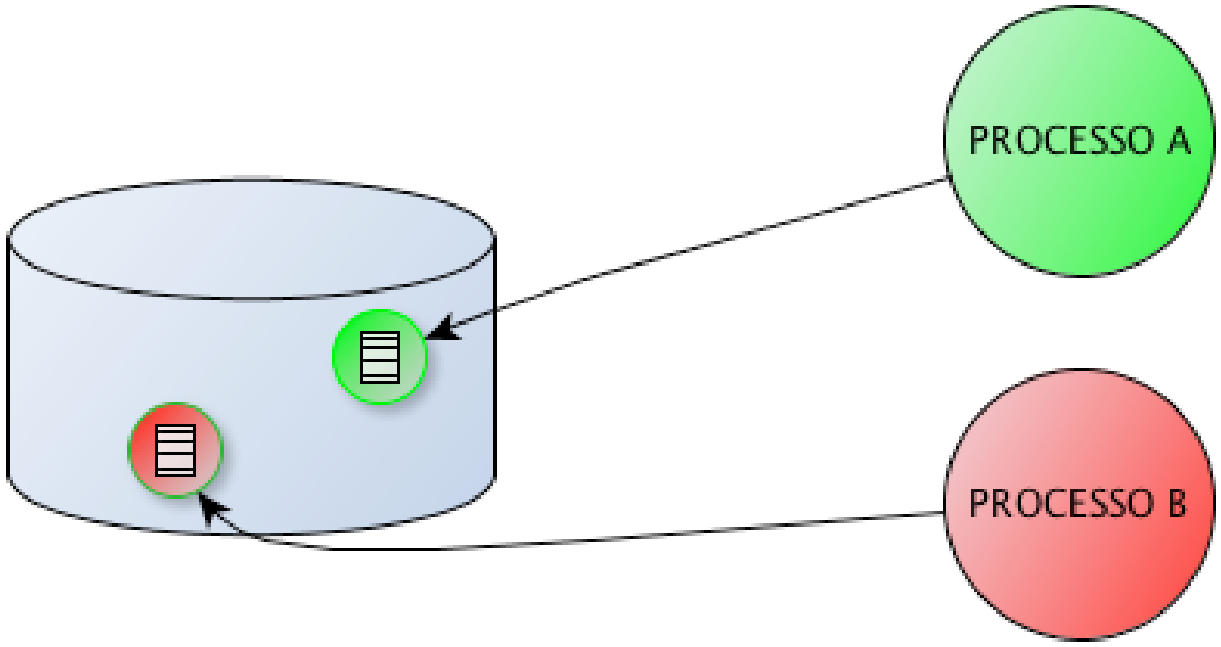
  (recursos de hardware e de software)
- É responsabilidade do sistema operacional controlar o acesso a estes recursos.
- Para muitas aplicações, um processo necessita acesso exclusivo não somente a um recurso, mas a vários.

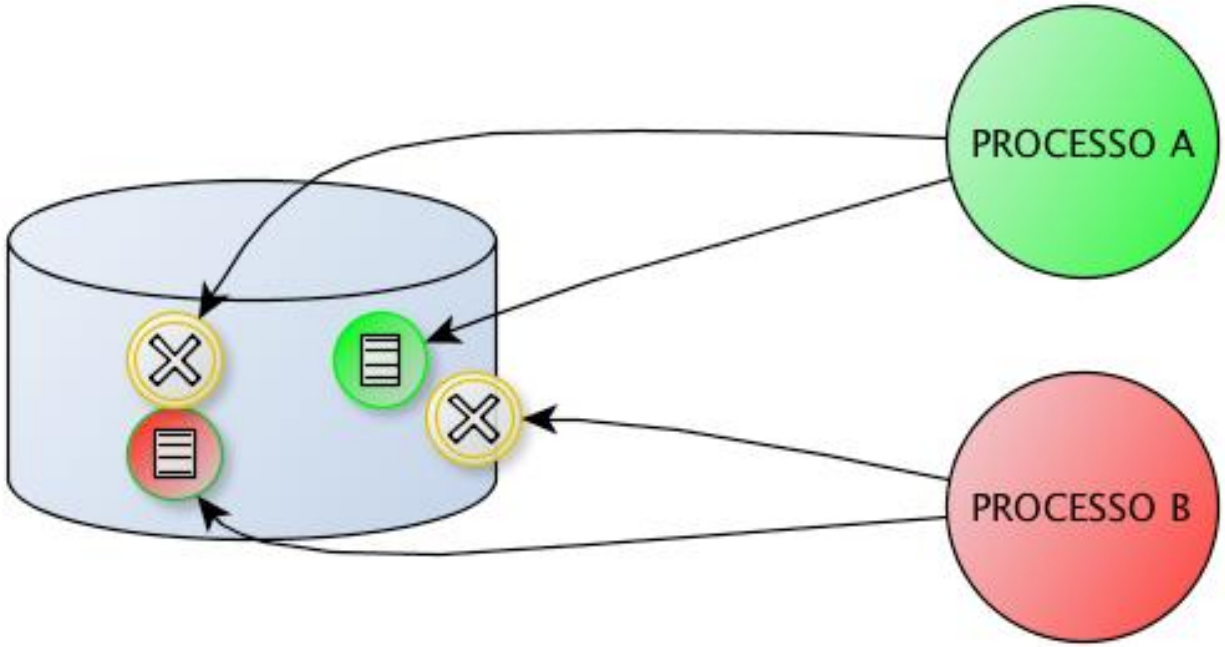
Exemplo




PROCESSO A
PROCESSO B
CD-ROM
IMPRESSORA


PROCESSO A
PROCESSO B
CD-ROM
IMPRESSORA


PROCESSO A
PROCESSO B


PROCESSO A
PROCESSO B

Recursos

- Um computador normalmente possui diferentes tipos de recursos que podem ser usados por processos;
- Recurso seria qualquer coisa que pode ser usada por um único processo em um certo instante;
- Para alguns recursos, várias instâncias idênticas podem estar disponíveis. Neste caso, qualquer uma pode ser usada para satisfazer a uma solicitação de uso do recurso;
- Os recursos podem ser preemptivos e não-preemptivos (obs.: não confundir o com o termo usado em escalonamento)

Recursos

- USO:
  - Processo solicita acesso ao recurso;
    - Se o recurso não está disponível, o processo é forçado a esperar (em alguns sistemas o processo é automaticamente bloqueado, em outros a solicitação falha e o processo deve esperar um pouco antes de tentar novamente)
  - Processo usa o recurso;
  - Processo libera o recurso.

Deadlock

- Um conjunto de processos está em deadlock se cada processo no conjunto está esperando um evento que somente outro processo no conjunto pode causar.

Condições para o Deadlock

- Condições para a ocorrência de Deadlock (Coffman, 1971):
  1. Exclusão mútua;
  2. Posse e espera;
  3. Não preempção de recursos;
  4. Espera circular;
- Todas estas condições devem estar presentes para que um deadlock possa ocorrer.
- Se somente uma destas condições estiver ausente, não existe deadlock.

Condições para o Deadlock

1. Exclusão mútua
   - Pelo menos um recurso deve ser alocado na modalidade não compartilhável; isto é, somente um processo de cada vez poderá usar este recurso.
2. Posse e espera
   - um processo deve estar de posse de pelo menos um recurso e aguardando para adquirir recursos adicionais que no momento estão a serviço de outros processos.

Condições para o Deadlock

3. Não preempção
   - Os recursos não podem ser objetos de preempção, isto é, um recurso só pode ser liberado pelo processo que o estiver retendo, de forma voluntária, após completar sua tarefa.
4. Espera circular
   - Deve existir um conjunto {P0, P1, P2, ..., Pn} de processos em espera de tal modo que P0 esteja aguardando por um recurso alocado por P1, P1 esteja aguardando por um recurso alocado por P2 ,... Pn-1 esteja aguardando por um recurso a serviço de Pn e Pn esteja aguardando por um recurso em utilização por P0.

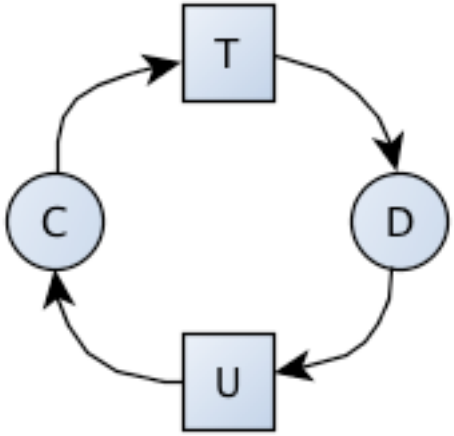
Modelagem de Deadlock

- Grafo de alocação de recursos
  - Conjunto de vértices (V) e arestas (A)
  - Vértices: processos ou recursos
  - Arestas: quando orientadas do processo Pi para um recuso Rj (Pi→Rj) significa que o processo Pi solicitou uma instância do recurso Rj e no momento está aguardando pelo recurso. Quando orientada no sentido contrário, ou seja, do recurso Rj para o processo Pi (Rj→Pi), siginifica que o processo está de posse do recurso Rj.

Modelagem de Deadlock







Processo A possui o recurso R

Processo B aguarda pelo recurso S

DEADLOCK

Pode-se demonstrar que se o grafo não contiver ciclos, nenhum processo no sistema estará em Deadlock. Se o grafo contiver um ciclo, poderá existir um Deadlock.

Estratégias para lidar com Deadlock

Em geral, quatro estratégias são usadas para lidar com deadlocks:

1. Ignorar completamente o problema e fingir que nunca ocorrerão deadlocks no sistema (usada pela maioria dos SOs);
2. Permitir ao sistema entrar em estado de deadlock, detectando-o e recuperando o sistema;
3. Evitar dinamicamente através da alocação cuidadosa de recursos;
4. Prevenção, estruturalmente negar uma das quatro condições necessárias

O Algoritmo do avestruz

- Trata-se da abordagem mais simples: esconder a cabeça na areia e fingir que não existe o problema.
- Reações diferentes
  - Matemáticos acham totalmente inaceitável e dizem que os deadlocks devem ser prevenidos a todo custo;
  - Engenheiros desejam saber com que frequência o problema ocorre e o quão sério são as suas consequências.

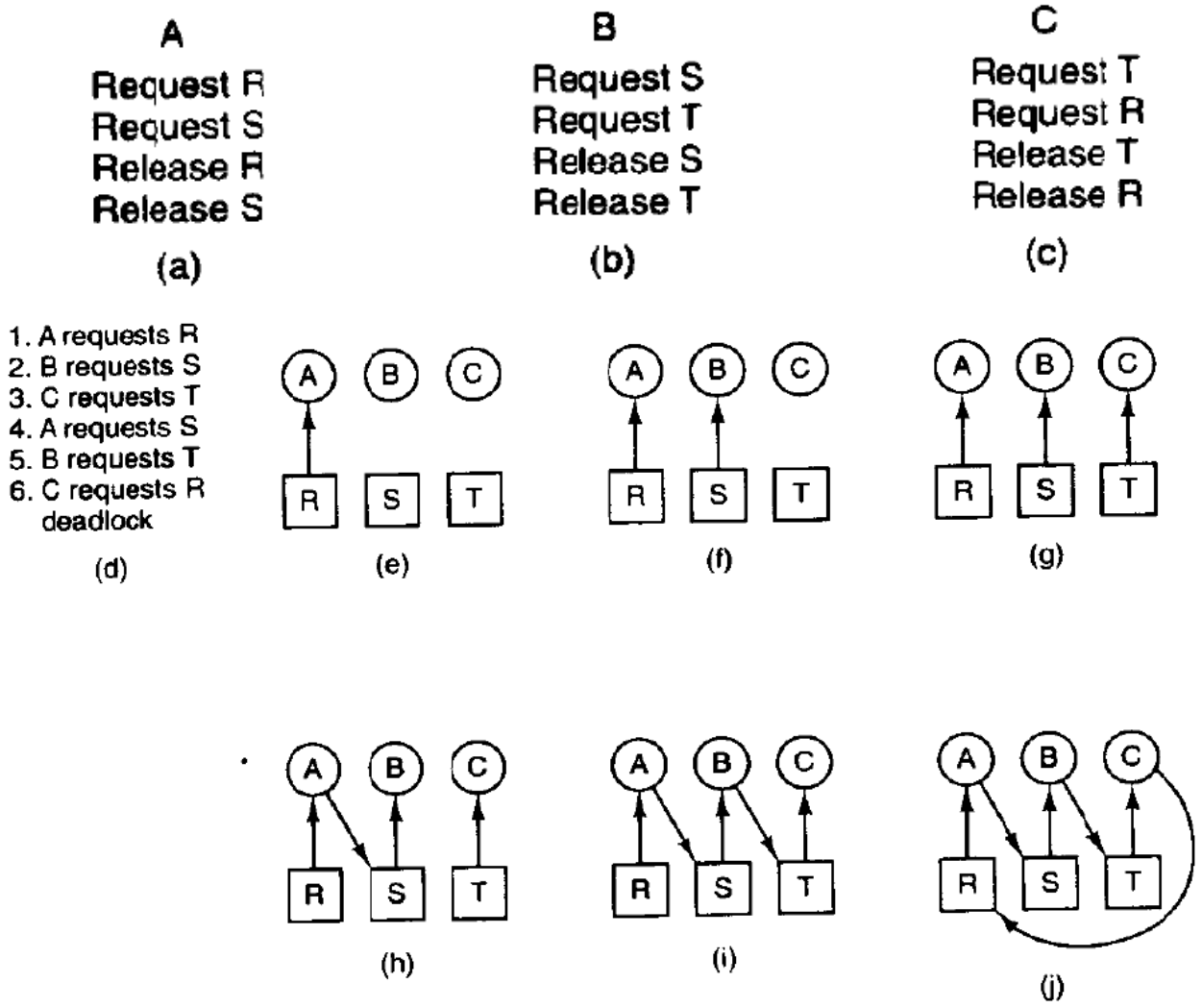
Detecção e recuperação

- Quando esta técnica é usada, o sistema não faz nada exceto monitorar as requisições e liberações de recursos.
- Cada vez que um recurso é liberado ou requisitado, o grafo de recurso é atualizado e verificado para ver se existe algum ciclo.
- Se um ciclo existe, um dos processo no ciclo é finalizado. Se isto não eliminar o deadlock, outro processo é finalizado e assim por diante até o ciclo não existir mais no grafo de recursos.
- Outro método mais simples seria não manter o grafo de recursos atualizado, mas ao invés, periodicamente verificar para ver se existem processos que tenham estado bloqueados por muito tempo (ex.: 1h). Neste caso, este processos seriam finalizados e seus recursos alocados liberados.

Prevenção de deadlocks

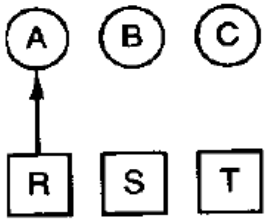
- Impor restrições aos processos de tal forma que o deadlock seja estruturalmente impossível.
  - Eliminar a exclusão mútua
    - (ex.: criar uma fila de impressão, com um processo daemon)
  - Prevenir processos que possuam um recurso de requisitar outros recursos
    - Um processo deve solicitar todos os recursos no inicio da execução.
  - considerar que todos os recursos são preemptivos
  - Eliminar a espera circular.
    - Numerar os recursos e impor aos processos que estes somente podem solicitar os recursos em ordem crescente.

Evitar Deadlocks

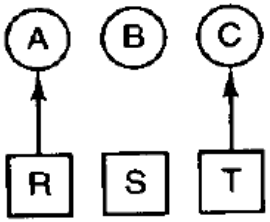


1. A requests R
2. C requests T
3. A requests S
4. C requests R
5. A releases R
6. A releases S
   no deadlock

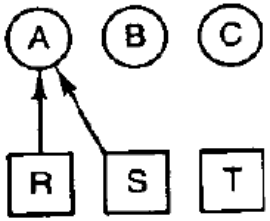
(k)



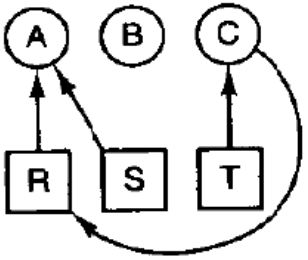
(l)



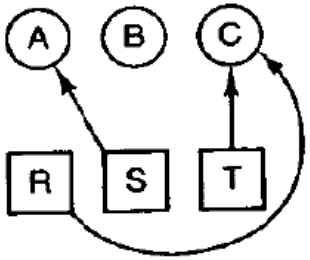
(m)



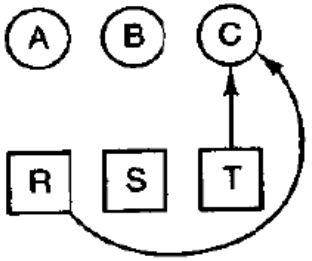
(n)



(o)



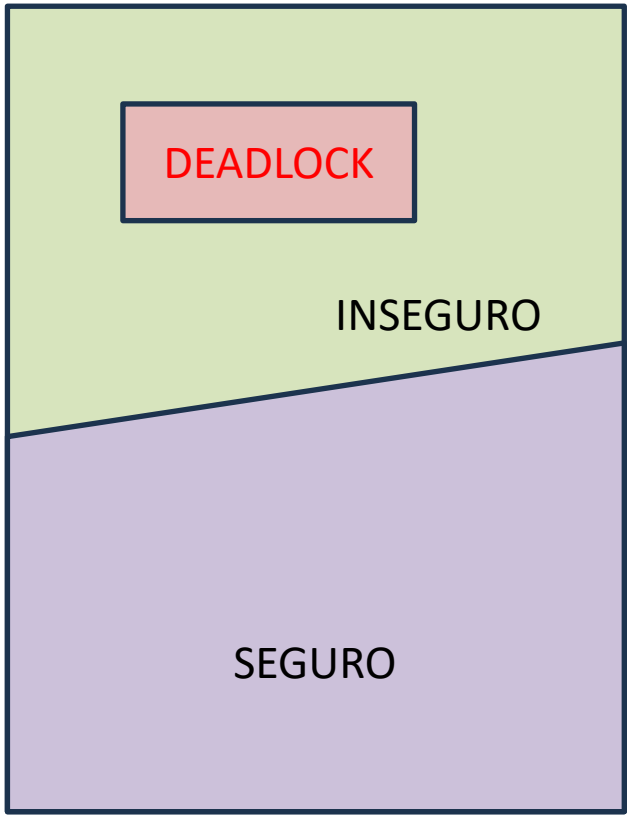
(p)



(q)

Estado seguro

- Um estado é seguro se o sistema puder alocar recursos a cada processo (até o seu máximo) em alguma ordem e ainda evitar o deadlock.
- Formalmente, um sistema está em estado seguro somente se houver uma sequencia segura.
- Sequencia segura: <P1, P2, ..., Pn> é uma sequencia segura para o esta de alocação atual se, para cada Pi, os recursos que Pi ainda possa precisar possam ser satisfeitos pelos recursos disponíveis mais os recursos mantidos por todo Pj, com j < i. Nessa situação, se os recursos mantidos por todo Pj



Algoritmo do banqueiro

- Algoritmos que evitam deadlocks são baseados em informações adicionais sobre a utilização dos recursos por parte dos processos.
- Um dos algoritmos mais simples solicita que o processo informe o número máximo de recursos de um mesmo tipo que possa vir a usar. Este número não deve exceder o total de recursos disponíveis no sistema.
- Quando o processo faz uma solicitação de recursos, o sistema deve determinar se a alocação destes recursos irá levar a um estado seguro.
- Se for este o caso, o recurso é alocado, do contrário, o processo deverá esperar algum outro processo liberar recursos.

exemplo

- Considerar um sistema com 12 unidades de fitas e 3 processos: P0, P1 e P2. O processo P0 necessita de 10 unidades de fita, o processo P1 pode necessitar de 4 e o processo P2 de até 9. Suponha que em um instante t0, o processo P0 alocou 5 unidades de fita, o processo P1 2 e o processo P2 2.
- Ao analisar este cenário, chegamos a conclusão que o sistema está em um estado seguro. A sequencia P1, P0 e P2 satisfaz a condição de segurança.

  Há 3 fitas livres e P1 só precisa de mais 2. Assim que P1 termina, devolve 4 → passam a ser 5 livres, suficientes para atender P0 (precisa de 5). P0 terminando devolve mais 10 → ficam 10 livres, suficientes para P2 (precisa de 7).
- Suponha que no instante t1, o processo P2 solicitou e alocou mais uma unidade de fita. O sistema agora não está mais no estado seguro (não existe sequencia segura)